

RAG PDF Study Assistant

What I tested, what I found, and why I chose chunk size 1500, overlap 300, K = 10

Shivam Kumar Choudhary | October 2026 | Code: github.com/Shiva123-del/pdf-study-assistant | Live demo: pdf-study-assistant-5rgb.onrender.com

The short version. I built an app that answers questions about a PDF book and cites the pages it used. Then I measured it properly instead of trusting how it felt. I ran 30 test questions on a computer-science textbook, compared 9 retrieval settings, used a second, independent AI model to grade the answers, and tried to break the system with 8 stress tests.

Chosen setup: split the book into pieces of **1500 characters** with **300 overlap**, and give the AI the **10 best-matching pieces** per question.

Why: it put the right page nearer the top of the results than any other setting (**MRR 84.17%**), found the right page for **93.33%** of questions (tied for best), and needs **63% fewer pieces** than the 500-character setting, so the book indexes about twice as fast.

1. What the app does

You upload a PDF. The app reads it page by page, cuts the text into small overlapping pieces called **chunks**, and stores them in a searchable index. When you ask a question, it finds the chunks most likely to contain the answer, hands only those to a language model (GPT-4.1-mini), and returns an answer with page citations. It can also turn the book into quizzes.

Two settings control how well this works, and they are what I tested: **chunk size** (how big each piece is) and **K** (how many pieces are retrieved per question).

2. How I tested it

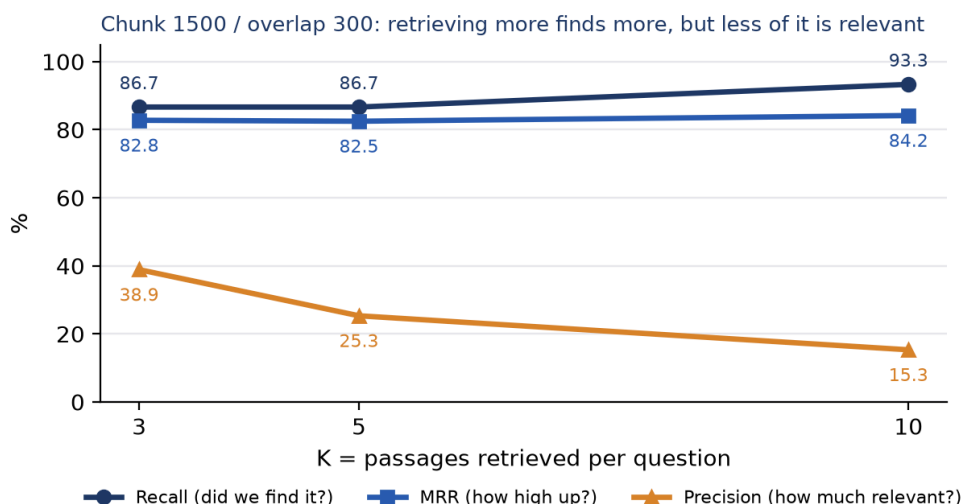
- **The benchmark.** 30 questions written from the OpenStax *Introduction to Computer Science* textbook (algorithms, binary search, hardware, data structures, trees, heaps, Big O and more). For each question I recorded which page holds the answer.
- **Retrieval tests (no AI cost).** Does the right page come back? Run for every combination of 3 chunk sizes and 3 values of K.
- **Answer-quality tests.** A separate, stronger model (GPT-5.6-sol) graded each generated answer from 1 to 5. Using a different model as the judge avoids the system marking its own homework.
- **Citation, robustness and speed tests.** Do the cited pages really support the claims? Does it survive typos, rephrasing, prompt-injection and off-topic questions? How long does a response take?

3. The measurements in plain words

Term	What it means
Recall@K	Out of all questions, how often did the page with the answer appear in the top K results? Higher is better. This is "did we find it at all?"
Precision@K	Of the K results returned, what share came from a relevant page? It falls as K grows, because there is only so much relevant material to find.
MRR	Mean Reciprocal Rank. Rewards putting the right page near the top: rank 1 scores 1.0, rank 2 scores 0.5, rank 3 scores 0.33. This is "how high up was it?"
Faithfulness	Judge score (1 to 5): does the answer stick to the retrieved text, or does it add things the text does not say?
MMR retrieval	Maximal Marginal Relevance. Picks passages that are relevant <i>and</i> different from each other, so the answer is not built from five copies of the same paragraph.

4. Finding the trade-off: coverage versus precision

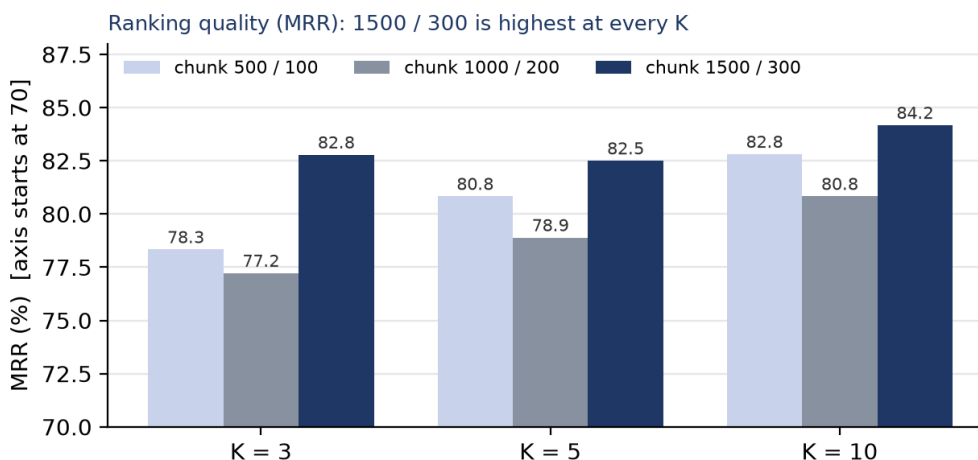
Retrieving more passages helps the system find the answer, but more of what it returns is irrelevant. This is the central trade-off in any retrieval system, and the data shows it clearly:



Going from K = 3 to K = 10 raised recall from 86.67% to 93.33%, while precision dropped from 38.89% to 15.33%. I accept the lower precision because the language model can ignore irrelevant passages, but it cannot use a passage that was never retrieved. A reranker (section 9) is the proper fix for the extra noise.

5. Choosing the chunk size

I compared 500/100, 1000/200 and 1500/300 (chunk size / overlap in characters) at K = 3, 5 and 10:



Chunk / overlap	K	Recall	Precision	MRR	Chunks in book	Index time
500 / 100	10	93.33%	23.33%	82.81%	6,340	~66 s
1000 / 200	10	93.33%	15.00%	80.83%	3,397	~38 s
1500 / 300	10	93.33%	15.33%	84.17%	2,349	~28 s

What this says. At K = 10 all three reach the same recall. 1500/300 ranks the right page best (MRR 84.17%) and is the cheapest to index. 500/100 has the best precision at every K, so it remains the better choice if keeping irrelevant text out matters most. I chose 1500/300 because ranking quality and indexing cost mattered more for this app, and, in my judgement, larger chunks keep a whole explanation together instead of splitting it across pieces.

6. How good are the answers?

Answer quality was graded on all 30 questions with the 1000/200, K = 5 setup. The 1500/300, K = 10 run was cut short at 16 of 30 questions when my API credit ran out. Its partial scores are shown for reference and are **not** counted as a finished benchmark.

Judge score (1 to 5)	Full run: 1000/200, K=5 (30/30)	Partial run: 1500/300, K=10 (16/30)
Faithfulness (sticks to the text)	4.73	4.75
Answer relevancy (answers the question)	4.97	5.00
Correctness	4.87	4.81

The two runs are very close, so there is no sign that the larger K hurts answer quality, but a full 30-question run of the chosen setup is still needed to confirm it.

Citations

Coverage 4.80/5, correctness 4.77/5, completeness 4.63/5, and **26 of 30 answers (86.67%)** had citations the judge considered fully supported. The main failure was **over-extension**: an answer was broadly right but added a detail the cited page did not state. Being correct is not the same as being backed by the source, and this is the weakness I most want to fix.

7. Does it break easily? And how fast is it?

Robustness: 8 of 8 tests passed (4.88/5). Rephrased questions, typos, very short and very long queries, off-topic questions (it did not invent an answer), and a prompt-injection attempt were all handled. The one soft spot was time-sensitive questions (4.0/5): a static textbook cannot know what is "latest" today, which is a limit of the data rather than a bug.

Speed (1000/200, K=5 setup): average response 1.93 s, 95% of responses under 2.77 s. Retrieval takes about 0.48 s of that; generation takes about 1.46 s. Each question used about 1,136 tokens on average.

8. What these numbers do not show

- **Small test set.** With 30 questions, one question is worth 3.33 percentage points. My first run scored 96.67% recall at K = 10 for 1000/200 and the later ablation scored 93.33% for the same setting: a one-question difference. Treat gaps of a few points between configurations as suggestive, not decisive.
- **One textbook.** Results may differ on other documents, such as scanned or table-heavy PDFs.
- **Page-based precision is approximate.** It counts a result as relevant if it comes from the right page, and some of my page labels were imperfect, so absolute precision is understated. The comparison between settings is more reliable than the raw values.
- **Speed was measured at K = 5.** K = 10 with 1500-character chunks sends roughly three times as much text to the model as K = 5 with 1000-character chunks, so response time is likely longer. I have not re-measured it.
- **An AI judged the answers.** That is useful and consistent, but it is not a human audit.

9. What I would do next

1. **Add a reranker** after retrieving 10 passages, to keep the recall and cut the noise.
2. **Verify citations claim by claim**, so a detail the page does not support gets removed or flagged.
3. **Let the system say "I don't know"** when the evidence is weak.
4. **Re-run the full 30-question answer evaluation** on the chosen 1500/300, K = 10 setup, and re-measure its latency.
5. **Expand the benchmark** to more documents and more questions, with human-checked labels.

10. Bottom line

The assistant finds the right page 93% of the time at K = 10, ranks it near the top, answers with a judge-rated quality above 4.7 out of 5, survives every stress test I threw at it, and responds in about two seconds. The chosen setting was picked from measured data, and I have written down where the data is thin. The evaluation scripts and raw result files are saved with the project, so every figure here can be re-run.